

# From Detection to Action: Blockchain-Governed Agentic AI for Autonomous Network Defense

1<sup>st</sup> MD Shariful Islam

*Dept. of Computer Science*

*University of Northern British Columbia*

Prince George, British Columbia

islam5@unbc.ca

2<sup>nd</sup> Sajal Saha

*Dept. of Computer Science*

*University of Northern British Columbia*

Prince George, British Columbia

sajal.saha@unbc.ca

**Abstract**—Agentic AI can automate network defense, but allowing AI agents to modify live enterprise networks introduces a critical trust problem: generated actions may be incorrect, altered, or inconsistent with security policy. We propose a unified end-to-end architecture that integrates intrusion detection, explainable attribution, policy-grounded mitigation generation, and blockchain-based action enforcement across distributed enterprise network domains. Vertical Federated Learning enables Access, Perimeter, and Endpoint domains to collaboratively detect attacks without sharing raw features, preserving data locality while maintaining performance comparable to centralized learning. SHAP identifies the domain contribution to each alert and guides a retrieval-augmented generation pipeline grounded in ten authoritative cybersecurity frameworks to produce domain-specific mitigation plans. Before execution, security actions are validated against a blockchain-stored whitelist derived from industry standards and established cybersecurity knowledge. Each generated action is deterministically checked against the detected attack type and the corresponding authorized action policy. Only actions that satisfy both checks are released for execution, while their outcomes are recorded on-chain to provide immutable and auditable enforcement. This design separates probabilistic AI reasoning from deterministic enforcement, preventing unapproved or modified actions from reaching the network even when the generated plan is unreliable or has been tampered with. The resulting architecture provides an accountable detection-to-action pipeline in which blockchain serves as the trust and enforcement layer for autonomous network defense.

**Index Terms**—Vertical Federated Learning, Agentic AI, Large Language Models, Retrieval-Augmented Generation, Blockchain Trust Enforcement, Enterprise Network Intrusion Detection, Smart Contracts, Distributed Network Security

## I. INTRODUCTION

The rapid growth of distributed enterprise networks, cloud services, edge devices, and heterogeneous computing environments has significantly increased the complexity of cybersecurity. Modern network attacks can span multiple domains and generate diverse observations across network, perimeter, and endpoint layers, making effective intrusion detection increasingly dependent on combining complementary security information. However, raw security telemetry is often distributed across organizational boundaries and cannot be freely shared because of privacy, data ownership, regulatory, and security constraints [1]–[3].

Artificial intelligence (AI) and machine learning (ML) have become important for identifying complex attack patterns and

supporting automated security analysis. Federated Learning (FL) enables collaborative model training without requiring direct exchange of raw data, while Vertical Federated Learning (VFL) is particularly suitable when different parties hold complementary features of the same security events [4], [5]. However, distributed detection alone provides limited insight into why an event is classified as malicious and does not directly determine how the detected threat should be addressed. Explainable attribution is therefore important for identifying the evidence contributing to a detection decision and supporting trustworthy downstream security reasoning.

Recent advances in Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and Agentic AI provide opportunities to extend cybersecurity systems from detection toward contextual reasoning and autonomous response. LLMs can interpret complex security events, RAG can ground reasoning in external security knowledge, and Agentic AI can plan and execute security actions [6]–[9]. Security decisions can further be grounded in established frameworks such as MITRE ATT&CK [10] and NIST SP 800-53 [2]. However, allowing agents to directly execute security actions introduces additional risks, including incorrect reasoning, prompt manipulation, compromised agents, and unauthorized actions.

This creates a fundamental trust requirement between autonomous reasoning and security execution. Blockchain and smart contracts can provide a tamper-resistant and independently verifiable governance layer for agent coordination, authorization, provenance, and policy enforcement [11]–[14]. Nevertheless, existing approaches do not jointly connect privacy-preserving intrusion detection and explainable attribution with grounded mitigation planning and verifiable security-action execution. In particular, an autonomous security system should ensure that an action is not only permitted by policy, but also corresponds to the mitigation plan authorized for the specific detected attack.

Motivated by these challenges, we propose a unified framework that integrates VFL-based intrusion detection, feature-level attribution, RAG-grounded LLM reasoning, Agentic AI planning, and blockchain-governed security enforcement to enable privacy-preserving collaborative intrusion detection and trusted autonomous response. The main contributions of this work are as follows:

- A unified end-to-end architecture from detection to mitigation for multi-domain enterprise intrusion response, in which VFL, SHAP-conditioned RAG, and blockchain enforcement are coupled rather than stacked as optional modules.
- Empirical evidence that a three-party VFL detector can approach centralized accuracy on CIC-style traffic while keeping Access, Perimeter, and Endpoint features local.
- An apply-time enforcement evaluation in which unauthorized actions and in-cycle plan substitutions are rejected, and accepted actions leave an on-chain audit record.

## II. RELATED WORK

The use of artificial intelligence (AI) and machine learning (ML) in cybersecurity has gained increasing interest due to their ability to identify complex attack patterns and adapt to evolving threats. This section reviews the literature across four key areas: federated learning for collaborative security, LLM- and RAG-based cybersecurity reasoning, blockchain-based trust and security enforcement, and agentic AI for autonomous security operations. Recent studies have explored these technologies for privacy-preserving detection, knowledge-grounded security reasoning, trusted coordination, and autonomous threat response, while also highlighting challenges related to explainability, governance, and the safe execution of autonomous actions. The review further identifies limitations and research gaps that motivate the proposed framework.

### A. Federated Learning in Cybersecurity

Federated learning (FL) supports collaborative intrusion detection without sharing raw telemetry, and vertical FL (VFL) is used when parties hold complementary features of the same events. Liu *et al.* [4] map VFL across effectiveness, efficiency, and privacy, and introduce VFLow to balance communication, computation, privacy, effectiveness, and fairness. Folino *et al.* [5] propose a scalable VFL framework for privacy-preserving cybersecurity analytics, but without feature-level attribution, which limits interpretability for downstream LLM reasoning and automated response.

Alazab *et al.* [15] present the first comprehensive survey on federated learning for cybersecurity, systematically categorizing its applications across authentication, privacy preservation, trust management, and attack detection alongside real-world deployment cases and open research challenges. Talpini *et al.* [16] proposed the SHIELDED architecture, a distributed federated learning framework integrating network-aware orchestration and SHAP-based anomaly detection, achieving 99.05% accuracy; however, it lacks LLM/RAG-grounded reasoning and automated security enforcement, limiting its capability for evidence-driven and adaptive response. Amin *et al.* [17] proposed an XGBoost–LightGBM FL framework for SDN threat detection and adaptive mitigation, achieving 96.3% accuracy; however, its conventional FL design does not address heterogeneous feature ownership or provide feature-level attribution and LLM/RAG-grounded reasoning. Yu, Han *et al.* [18]

taxonomize VFL privacy threats and defenses by life-cycle stage, while Ye *et al.* [19] survey VFL from effectiveness, security, and applicability. Neither connects VFL detection to grounded mitigation or pre-execution action enforcement.

Overall, existing studies establish the effectiveness of FL and VFL for privacy-preserving cybersecurity analytics, but they largely lack integrated feature-level explainability, grounded LLM reasoning, and trustworthy automated security enforcement, motivating a unified framework that connects detection, attribution, reasoning, and response.

### B. LLM- and RAG-Based Cybersecurity Reasoning

Agentic AI has emerged as a promising approach to cybersecurity by enabling autonomous threat detection, decision-making, and incident response. Large Language Model (LLM)-based agents extend conventional security analytics by introducing reasoning, planning, memory, and tool interaction [6]. Li and Zhu [7] formulate Agentic AI for cyber resilience as a detect–reason–act paradigm. Fu *et al.* [9] proposed FeRAG, a federated RAG-based LLM framework for privacy-preserving cyber threat detection; however, it lacks feature-level explainability and automated, blockchain-governed security enforcement. Kshetri [20] highlights that increasing autonomy also introduces risks such as adversarial manipulation, loss of human oversight, and expanded attack surfaces, emphasizing the need for effective governance and monitoring. Alsuaiket *et al.* [21] proposed ZeroDay-LLM, combining distributed FL, BERT-based threat reasoning, and DQN-based adaptive response to achieve 97.8% accuracy and 95.7% zero-day detection; however, it lacks VFL-based attribution, RAG-grounded reasoning, and blockchain-governed enforcement. However, increased autonomy also introduces new security risks. Lazer *et al.* [22] and Chhabra *et al.* [23] identify threats involving agent integrity, accountability, and cross-domain attack propagation. These concerns become more significant when agents can invoke operational tools, since erroneous reasoning, prompt injection, or compromised agents can translate directly into consequential actions.

### C. Blockchain-Governed Trust and Security Enforcement

As autonomous AI agents increasingly perform security-sensitive actions, trusted execution becomes essential to prevent unauthorized or unintended responses. Blockchain provides a suitable governance layer through tamper-resistant records, decentralized verification, provenance, and smart-contract-based policy enforcement. Recent studies have therefore explored blockchain for trustworthy multi-agent coordination, agent verification, policy enforcement, and autonomous execution.

Karim *et al.* [11] surveyed AI-agent–blockchain integration, highlighting decentralized reasoning and smart-contract governance while leaving VFL-based detection, RAG-grounded reasoning, XAI, and automated cybersecurity enforcement unaddressed. Pendli [24] reviewed blockchain-based security for agentic AI, highlighting its role as a trust and governance layer, but lacking an integrated VFL-based detection, XAI

attribution, RAG reasoning, and automated security enforcement framework. Chen et al. [12] proposed BlockAgents, a blockchain-enabled LLM multi-agent framework using Proof-of-Thought consensus, reducing poisoning degradation but lacking VFL, XAI threat attribution, RAG reasoning, and automated security enforcement. Jin et al. [13] proposed DeCoAgent, an LLM-based decentralized agent framework using few-shot prompting, RAG, and smart contracts, but lacking VFL-based detection, XAI threat attribution, and automated cybersecurity enforcement. Alqithami [25] surveyed agent-blockchain interoperability, proposing intent-based execution and verifiable policy enforcement, but lacking VFL-based detection, XAI threat attribution, RAG-grounded reasoning, and automated cybersecurity enforcement.

Chen et al. [26] introduced AgentChain, using PoCQ-based decentralized multi-agent consensus and stake reassignment to improve the trustworthiness of LLM-generated responses, reducing poisoning, backdoor, and jailbreak attack success rates to below 4%; however, it does not address distributed cyberattack detection or security-action enforcement. Syed et al. [27] integrated LLM reasoning, reinforcement learning, multi-agent coordination, and blockchain-based auditing for autonomous software supply-chain defense, but does not address VFL-based network intrusion detection, feature-level attribution, or event-specific mitigation-plan integrity.

Wickremasinghe et al. [28] investigated state verification in asynchronous blockchain-driven Agentic AI systems, focusing on state consistency and Byzantine-resilient verification, but does not address cyberattack detection or security-action authorization. Rahmati Tabrizi et al. [29] proposed BEAF, combining blockchain accountability with policy-as-code for high-assurance autonomous AI; however, its focus is clinical AI rather than network intrusion response. Gao et al. [30] proposed TrustAgentNet, using consortium blockchain to verify agent identities and skillsets, but it does not address attack-specific mitigation planning or plan integrity.

Fan et al. [31] proposed SecureVFL, a permissioned-blockchain VFL design that uses replicated secret sharing and Proof of Feature Sharing to protect multi-party training without homomorphic encryption; it does not enforce attack-specific mitigation actions. Singh et al. [32] implemented Zero Trust on Ethereum, using smart contracts as the policy engine to enforce MFA, RBAC, and just-in-time access, but without an AI planner or mitigation-to-action binding. Zhang et al. [33], Xu [34], Ressi et al. [35], and Nguyen Thanh et al. [36] further investigate blockchain for provenance, governance, trustworthy AI, and autonomous-agent ecosystems. Collectively, these approaches strengthen trust, access control, or training integrity, but they do not provide an integrated cybersecurity detection-to-action mechanism.

TABLE I: Comparison of related works on blockchain-governed, VFL-based, and agentic-AI cybersecurity frameworks.

| Paper (Year)                                                           | Domain / Application                                     | Core Mechanism                                                                        | Blockchain Type                    | Consensus / Protocol                                                    | Key Contribution                                                              | Main Limitation                                                        |
|------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Shajarian et al. [8], ReGAIN (2026)                                    | Network traffic analysis (ICMP / SYN flood)              | RAG with MMR sampling, cross-encoder re-ranking                                       | None                               | N/A                                                                     | Citation-grounded, hallucination-resistant explanations                       | 74–76% precision on ping flood; API latency limits real-time use       |
| Fu et al. [9], FeRAG (2026)                                            | Cyber threat detection in Internet-of-Energy             | Federated learning + RAG with vector DBs and RRF re-ranking                           | None                               | FL weighted averaging                                                   | FL privacy + RAG grounding; GPT-4o + FeRAG = 0.849 F1                         | No blockchain governance; detection only, no enforcement               |
| Chen et al. [12], BlockAgents (2024)                                   | Multi-agent problem-solving and planning                 | Proof-of-Thought (PoT): stake-based miner designation + debate voting                 | Abstracted                         | PoT with multi-metric evaluation prompts                                | Rewards highest-quality “thought” with accounting rights                      | Accuracy drops if malicious agents dominate miner role                 |
| Jin et al. [13], DeCoAgent (2024)                                      | Decentralized agent marketplace                          | Six LLM modules mapping NL → smart-contract calls                                     | Ethereum (Ganache / Remix)         | Standard Ethereum                                                       | LLM converts NL into contract calls for registration and task assignment      | No policy/whitelist enforcement; inconsistent LLM outputs              |
| Jan et al. [14], Blockchain-Monitored Agentic AI (2025)                | Agentic AI (healthcare, smart city, inventory)           | Perception → Conceptualization → Blockchain Governance → MCP Action                   | Permissioned (Hyperledger Fabric)  | Smart-contract validation (Action Registry, Policy Control, Evaluation) | End-to-end blockchain governance of LangChain agents before MCP execution     | No mitigation-to-action binding; no XAI                                |
| Alqithami [25], Autonomous Agents on Blockchains (2026)                | Agent-blockchain interoperability (DeFi, DAOs, treasury) | SLR of 317 works; 5-part taxonomy; Transaction Intent Schema + Policy Decision Record | Public (Ethereum, L2, ERC-4337)    | N/A (survey); policy-gated + MEV-aware execution                        | Threat model for agent-driven transactions; eval-suite proposal               | Position paper; abstractions not implemented or benchmarked            |
| Chen et al. [26], AgentChain (2026)                                    | Trustworthy multi-agent LLM QA (medical)                 | Decentralized council with Proof-of-Content-Quality (PoCQ)                            | Abstracted                         | PoCQ: VRF council + debate voting + stake incentives                    | Semantic consensus for stochastic LLMs; game-theoretic proofs                 | FT < 50% malicious; large councils show diminishing returns            |
| Wickremasinghe et al. [28], State-verification async blockchain (2026) | Agentic-AI orchestrators over async blockchains          | Timed automaton + BFT gossip + ZKP; disagreement as change-detection signal           | Async BFT (HoneyBadgerBFT-style)   | Async BFT + timed automaton + ZKP; $f = \lfloor (\omega - 1)/3 \rfloor$ | Solves 37.3% agent-state mismatch; ZKP-verified responses                     | Limited to state consistency; no policy or action-execution governance |
| Rahmati-Tabrizi et al. [29], BEAF (2025)                               | Clinical AI accountability (EU AI Act)                   | Stakeholder-Risk Matrix + policy-as-code; ZKP overlay                                 | Permissioned (Fabric + Idemix ZKP) | Raft (EtdRaft), 2-of-3 endorsement                                      | Maps regulatory liability to stakeholders; sub-250 ms p95 latency             | Not attack-focused; auditor UI/UX only partial                         |
| Gao et al. [30], TrustAgentNet (2025)                                  | Agentic AI networking (6G), skillset verification        | Chain of Skillset (CoS): skillset chain + training chains                             | Consortium (Hyperledger Fabric)    | Etdraft/Raft + FedAvg + MSP                                             | Verifiable skillset tags; 3-way security/performance/cost trade-off           | More consensus nodes increase latency, CPU, and overhead               |
| Fan et al. [31], SecureVFL (2025)                                      | Privacy-preserving VFL (banking)                         | Replicated Secret Sharing + novel (4,2)-sharing                                       | Permissioned                       | PoFS + DPoS committee                                                   | Addition-only sharing (no HE); 2-of-4 dropout tolerance; anonymous identities | Three-party base protocol fails if any party drops                     |
| Singh et al. [32], Blockchain-Enabled ZT for FinTech (2025)            | FinTech (insider threats, malware, APTs)                 | Ethereum contracts for MFA, RBAC, and JIT; blockchain as PE/PEP/policy store          | Public / consortium Ethereum       | Ethereum PoS + smart-contract policy execution                          | Immutable access control and tamper-proof audit trails; STRIDE-validated      | Higher latency and lower throughput; no AI-agent integration           |

Table I summarizes representative studies across these research directions. The comparison considers the Domain, core mechanism, blockchain type, consensus type, key contributions and limitations of each approach. This overview highlights that existing research has made substantial progress in individual components of autonomous cyber defense, but the components are generally investigated independently or in limited combinations.

To further distinguish the proposed architecture from existing approaches, Table II compares the capabilities provided by representative studies. We organize these capabilities into two complementary groups. The first is the *IDS/Intelligence Capability*, covering intrusion detection, explainability, RAG/LLM-based reasoning, and agentic action. The second is the *Governance and Protection Capability*, covering policy grounding, pre-execution protection, and plan integrity.

The comparison shows that existing approaches address important components of autonomous cyber defense, but no representative approach jointly connects privacy-preserving detection and explainability with grounded reasoning and blockchain-governed execution. Federated approaches primarily support collaborative intrusion detection, while RAG/LLM-based systems extend cybersecurity analysis with knowledge-grounded reasoning. Blockchain-based agentic systems, in contrast, focus mainly on coordination, trust, policy enforcement, provenance, or execution assurance. Consequently, the integration of these capabilities for autonomous intrusion response remains insufficiently addressed.

A key challenge is ensuring that an autonomous security action is both authorized and faithful to the mitigation decision generated for the corresponding attack. A policy whitelist can verify that an action is generally permitted, but cannot ensure that a compromised agent has not substituted another permitted action. Conversely, an immutable commitment can verify that a mitigation plan has not been modified, but does not establish whether the executed action is authorized for the detected attack. This work addresses these limitations by jointly verifying action authorization against the detected attack class and binding the executed action to the mitigation plan committed for the corresponding detection event. Blockchain therefore serves as a pre-execution governance layer that provides both policy-based authorization and mitigation-plan integrity for trusted autonomous security enforcement.

### III. END-TO-END: SYSTEM DESIGN

For trustworthy operational autonomous defense in a disaggregated enterprise network, we design a unified architecture structured into four static functional layer as illustrated in Fig. 1 : **(1) Detection Layer**, where distributed domain agents collect local network information and VFL collaboratively detects attacks without sharing raw data, while SHAP identifies the contribution of each domain; **(2) Reasoning and Planning Layer**, where the detection result and domain attribution are provided to a RAG-grounded LLM agent to generate policy-informed, domain-specific mitigation plans; **(3) Blockchain**

**Governance Layer**, where the generated plan is cryptographically committed and smart contracts verify action integrity and policy compliance before execution; and **(4) Execution and Monitoring Layer**, where validated actions pass through an execution gate, are dispatched to the appropriate network devices through API/gRPC/MCP interfaces, and are monitored by the SOC with human oversight for critical or rejected actions. Each network flow is processed across the system layers, progressing through the **Detect, Reason, Commit, and Apply** stages

#### A. Detection Layer (Detect)

The Detection Layer performs collaborative intrusion detection across the Access/ISP, Perimeter/IDS, and Endpoint/EDR domains using Vertical Federated Learning (VFL). Network attacks typically manifest across multiple enterprise domains, where each domain observes a distinct subset of the evidence associated with an aligned network flow. The Access/ISP domain captures connectivity and access-related characteristics, the Perimeter/IDS domain observes boundary traffic and intrusion indicators, and the Endpoint/EDR domain provides host-level behavioral information. These complementary feature subsets remain within their respective domains and are jointly exploited through VFL without centralizing raw telemetry. Each network agent utilizes a local feature extractor (Encoder)—a Multilayer Perceptron (MLP) model to process its specific subset of local features. Raw features remain within their originating domains throughout the detection process. Domain agents are indexed D1 (Perimeter/IDS), D2 (Access/ISP), and D3 (Endpoint/EDR), corresponding respectively to the feature blocks P, A, and E defined below. For each aligned flow  $i$ , features are split by domain as  $\mathbf{x}_i = \{\mathbf{x}_i^A, \mathbf{x}_i^P, \mathbf{x}_i^E\}$ . Each party keeps its own columns (23, 32, and 42 features) and never shares raw data.

A local MLP encoder (input  $\rightarrow 128 \rightarrow 64$ ) maps each slice to a 64-dimensional embedding  $\mathbf{h}_i^p = f_p(\mathbf{x}_i^p; \theta_p)$ ,  $p \in \{A, P, E\}$ . The Coordinator Agent receives only these embeddings. Its VFL Aggregator concatenates them,  $\mathbf{h}_i = [\mathbf{h}_i^A \parallel \mathbf{h}_i^P \parallel \mathbf{h}_i^E] \in \mathbb{R}^{192}$ , and a shared classifier ( $192 \rightarrow 128 \rightarrow K$ ) produces the attack label  $\hat{y}_i$ .

KernelSHAP then attributes  $\hat{y}_i$  over the 192-d fusion vector. Scores are summed inside each domain's 64-d block; the largest block names the responsible domain  $p_i^*$ .

The resulting pair  $(\hat{y}_i, p_i^*)$ , representing the detected attack label and responsible domain, forms the output of the Detection Layer and is forwarded by the Coordinator Agent to the Reasoning and Planning Layer for mitigation generation.

#### B. Reasoning and Planning Layer (Reason)

The Reasoning and Planning Layer takes the Coordinator's detection products—predicted attack class  $\hat{y}_i$ , confidence, and SHAP attributes  $p_i^*$  with per-domain and top-feature contributions—and turns them into a mitigation plan. It does not authorize execution. Three blocks in Fig. 1 implement the step: the RAG Pipeline, the LLM Agent, and Mitigation Plans.

TABLE II: Capability-based comparison of existing autonomous cyber defense approaches.

| Work                       | IDS / Intelligence Capability |     |         |                | Governance & Protection Capability |               |            |                |
|----------------------------|-------------------------------|-----|---------|----------------|------------------------------------|---------------|------------|----------------|
|                            | IDS                           | XAI | RAG/LLM | Agentic Action | Policy Grounding                   | Pre-Execution | Protection | Plan Integrity |
| Folino et al. [5]          | ✓                             | ×   | ×       | ×              | ×                                  |               | ×          | ×              |
| ReGAIN [8]                 | ✓                             | ×   | ✓       | ×              | ✓                                  |               | ×          | ×              |
| Fu et al. [9]              | ✓                             | ×   | ✓       | ×              | ✓                                  |               | ×          | ×              |
| BlockAgents [12]           | ×                             | ×   | ✓       | ✓              | ×                                  |               | ✓          | ×              |
| DeCoAgent [13]             | ×                             | ×   | ✓       | ✓              | ×                                  |               | ✓          | ×              |
| Jan et al. [14]            | ×                             | ×   | ✓       | ✓              | ✓                                  |               | ✓          | ×              |
| Alazab et al. [15]         | ✓                             | ×   | ×       | ×              | ×                                  |               | ×          | ×              |
| Talpini et al. [16]        | ✓                             | ✓   | ×       | ×              | ×                                  |               | ×          | ×              |
| Amin et al. [17]           | ✓                             | ×   | ×       | ×              | ×                                  |               | ×          | ×              |
| Alsuwaiket [21]            | ✓                             | ×   | ✓       | ×              | ×                                  |               | ×          | ×              |
| Alqithami [25]             | ×                             | ×   | ✓       | ✓              | ✓                                  |               | ×          | ×              |
| AgentChain [26]            | ×                             | ×   | ✓       | ✓              | ×                                  |               | ✓          | ×              |
| Syed et al. [27]           | ×                             | ×   | ✓       | ✓              | ×                                  |               | ✓          | ×              |
| Wickremasinghe et al. [28] | ×                             | ×   | ×       | ✓              | ×                                  |               | ✓          | ×              |
| BEAF [29]                  | ×                             | ×   | ×       | ✓              | ✓                                  |               | ✓          | ×              |
| TrustAgentNet [30]         | ×                             | ×   | ✓       | ✓              | ×                                  |               | ✓          | ×              |
| SecureVFL [31]             | ✓                             | ×   | ×       | ×              | ×                                  |               | ×          | ×              |
| Singh et al. [32]          | ×                             | ×   | ×       | ×              | ✓                                  |               | ✓          | ×              |
| Wu et al. [37]             | ×                             | ✓   | ×       | ×              | ×                                  |               | ×          | ×              |
| <b>Our Work</b>            | ✓                             | ✓   | ✓       | ✓              | ✓                                  |               | ✓          | ✓              |

a) *SHAP attributes as planning context.*: SHAP is computed in the Detection Layer over the 192-dimensional fused embedding (three 64-d domain blocks). What this layer receives is already structured evidence: which of Access/ISP, Perimeter/IDS, or Endpoint/EDR dominated the score, and which local features drove that block. For example, A **PortScan** alert with Perimeter  $\approx 45\%$  tells the planner to start at the inspection tier, not to invent a host-only story. These fields, with logits and confidence, are injected into the LLM prompt so recommendations track the detector’s drivers rather than generative assumptions.

b) *RAG Pipeline.*: The RAG Pipeline is the policy box in the architecture (“security policies + response knowledge”). It retrieves from a constrained corpus—NIST SP 800-53 Rev. 5 [2], NIST SP 800-207 [3], CIS Controls v8 [38], and MITRE ATT&CK [10]. Chunks (size 512, overlap 64) are embedded with all-MiniLM-L6-v2 and indexed in FAISS. Each event is queried with  $\hat{y}_i$ ,  $p_i^*$ , confidence, and the closed action list for that attack class; top- $k$  hits are reranked with a cross-encoder and the leading passages are passed forward with source metadata. Retrieval returns guidance and allowable primitives. It does not propose new actions.

c) *LLM Agent and Mitigation Plans.*: The LLM Agent (GPT-4o-mini) is the planning box. Its prompt has four parts: role and safety; structured context (prediction, SHAP, RAG); constrained reasoning (interpret the attack on all three domains, name which domain acts first, and scale intensity with confidence); and a deterministic JSON plan. Every action must come from the whitelist and must set `network_tier` to exactly one of Access / ISP, Perimeter / IDS, or Endpoint / EDR. The Mitigation Plans object—threat level, primary and supporting actions, evidence, execution priority, and overall reasoning—is the layer’s only output. That object is handed to the Blockchain Governance Layer (AgenticTrustRegistry.sol) for commit; the

Execution and Monitoring Layer applies it only if the gate agrees.

### C. Blockchain Governance Layer (Commit)

The Reasoning and Planning Layer is probabilistic. An LLM can emit a fluent action that is still harmful on a live path, and a poisoned or substituted plan can change between generation and dispatch. In blockchain, a smart contract is a program that lives on a ledger and executes exactly the rules written in its code. Callers submit transactions; the network orders them; every honest replica applies the same function to the same state. The contract cannot hallucinate those actions, and it cannot be talked into accepting them.

1) *Smart Contract-Based Policy Enforcement and Plan Anchoring*: AgenticTrustRegistry.sol (Solidity 0.8.20) is a permissioned registry with two jobs. First, a per-attack whitelist  $W[a, u]$  is seeded from policy and operational knowledge Table I. `isActionWhitelisted` admits only those mitigations that the playbook allows for attack class  $a$ . An action that is legal for DDoS but not for the predicted class fails here. Second, after the Mitigation Plans object is persisted off-chain, the backend canonicalizes payload  $P$  (identity, RAG context, structured plan and tiers) and writes only  $c = \text{SHA256}(P)$  via `anchor` [9], [32]. Telemetry, SHAP tables, and raw LLM text never go on-chain. `already_anchored` rejects overwrites, so  $c$  is write-once. Later readers use `getCommitment`. Unauthorized edits to the off-chain plan are detected because a re-hash no longer matches  $c$ ; confidentiality is preserved because the ledger holds a digest, not the data.

Immutability in this architecture is therefore not “the blockchain stores the network.” It is three enforceable facts: the allowed action set for each attack is on-chain and not promptable; the plan for this cycle has a single committed digest; and any later apply must bind to both.

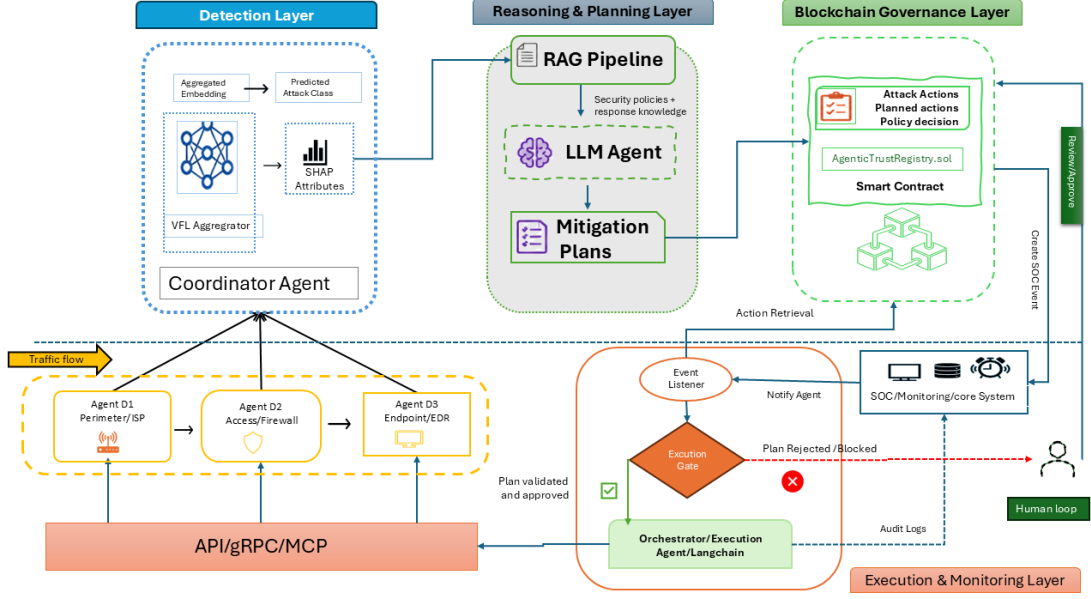


Fig. 1: End-to-end architecture. Domain agents feed a privacy-preserving detector, which hands off to a policy-grounded reasoning layer that drafts a mitigation plan. The plan is hashed and anchored on-chain before execution, then admitted only if it passes both a policy whitelist and a plan-binding check; rejected actions go to operator review, approved ones are dispatched.

2) *Blockchain-Triggered SOC/SIEM Coordination*: Once a plan is anchored, the contract is the trigger—not the actuator. A commitment event (*TrustAnchored*) is a tamper-evident signal that a validated proposal exists. That signal can be consumed by whatever operational fabric the enterprise already runs: a SOC console, a SIEM correlation rule, a ticket queue, or an equivalent monitoring bus. The architecture does not require a vendor-specific SOC. Any subscriber that can watch the registry (or the backend’s “create SOC event” path in Fig. 1) is sufficient.

3) *Execution Gate and Monitored Actuation*: The Execution and Monitoring Layer is that subscriber. An Event Listener performs action retrieval against the anchored record and notifies the Orchestrator Agent (execution agent / LangChain control path). The Orchestrator does not invent a new plan. It presents the committed actions to the Execution Gate: whitelist for this attack, binding to  $P$ , then *applyAction* or block. Audit logs and alerts return to the same SOC/SIEM so operators see both accepted actuations and rejected ones (human loop). In short, the smart contract freezes what may be done; the SOC/SIEM notices that a frozen plan exists; the Orchestrator Agent is the component that carries that notice into Apply.

#### D. Execution and Monitoring Layer(Apply)

The Execution and Monitoring Layer is the only place a planned action may touch the live network. As mentioned in Fig. 1, the SOC or core monitoring system is notified, an Event Listener retrieves the on-chain record, an Execution Gate compares that record with policy, and only then may the

TABLE III: Per-attack action whitelist  $W$  at Apply time. Labels follow CICIDS-2017 [1]; actions are a constrained catalog, not free-form LLM text.

| Attack $a$              | Allowed actions (excerpt)                 |
|-------------------------|-------------------------------------------|
| BENIGN                  | log incident, monitor traffic             |
| DDOS / DOS              | limit rate, enable scrubbing, block IP    |
| SSHPATATOR / FTTPATATOR | fail2ban block, lock account, enforce MFA |
| PORTSCAN                | tarptit scan, harden ports, update ACL    |
| WEBATTACK               | apply WAF, virtual patch, isolate service |
| BOT                     | captcha / js challenge, reputation filter |
| OTHERS                  | log incident, isolate service, update ACL |

Orchestrator Agent dispatch. The execution gate follows the Algorithm 1.

#### Algorithm 1 Commit and Apply in AGENTICTRUSTREGISTRY

```

0: procedure COMMIT(agentKey, reportKey, c)
0:   require  $c \neq 0$  and not already committed
0:    $commitments[agentKey][reportKey] \leftarrow c$ 
0:   emit TRUSTANCHORED
0: end procedure
0: procedure APPLY( $a, u, \tau, agentKey, reportKey$ )
0:   require  $\tau$  matches tier in  $plan(P)$  {restore Detect / Reason tier}
0:   require  $W[a, u] = 1$  {policy ground}
0:   require  $commitments[agentKey][reportKey] \neq 0$ 
0:   require not already applied
0:   off-chain:  $u \in plan(P)$  and  $c_{chain} = c_{rehash}$  {plan binding}
0:   mark applied; emit ACTIONAPPLIED
0:   dispatch  $u$  to domain  $\tau$ 
0: end procedure

```

a) *Event Listener*.: After the Blockchain Governance Layer commits a plan, it can raise a SOC event. That event is not vendor-specific: a SOC, a SIEM, or any equivalent core monitoring bus can subscribe. The Event Listener is the agentic receiver in this layer. It is notified by the SOC/core (“Notify Agent” on the figure) and pulls the corresponding actions from the ledger (“Action Retrieval”): the predicted attack class, the committed plan, and the per-attack whitelist already stored in `AgenticTrustRegistry.sol`.

b) *Execution Gate*.: The listener hands that bundle to the Execution Gate (the diamond on the figure). The gate does not invent mitigations. It compares three facts that already exist: (i) whether action  $u$  is on the whitelist  $W$  for this attack (Table III); (ii) whether  $u$  and its domain  $\tau$  (Access, Perimeter, or Endpoint) are the same actions Reason wrote into plan  $P$ ; (iii) whether a re-hash of the off-chain payload still matches the on-chain digest `getCommitment`. In compact form,

$$W[a, u] = 1 \wedge u \in \text{plan}(P) \wedge c_{\text{chain}} = c_{\text{rehash}}. \quad (1)$$

An LLM action that is only plausible, or that is legal for a different attack, fails the whitelist. A still-whitelisted action copied from another cycle fails plan binding. A silently edited payload fails the digest check.

c) *Reject path and human loop*.: If the gate does not match, the plan is marked rejected/blocked. The flow goes to the human loop on the figure, not to the network. Operators see the event in the SOC/core. Every such outcome is written to the audit log store in that same core system, so a denial is as visible as an apply. If an operator later reviews and approves, that approval is recorded back on the blockchain (Review/Approve into the Governance Layer). The SOC/core is triggered again and notifies the Event Listener, which re-enters the same retrieve–compare path. Approval does not skip the gate; it re-anchors a decision the contract can still check.

d) *Success path and Orchestrator Agent*.: If the gate validates and approves the plan, control passes to the Orchestrator / Execution Agent (LangChain on the figure). That agent calls the domain-specific interface—API, gRPC, or MCP—for the tier named in the plan, and the corresponding domain agent ( $\mathcal{D}_1$ ,  $\mathcal{D}_2$ , or  $\mathcal{D}_3$ ) carries out the action. `applyAction` records the on-chain receipt and refuses duplicates. Dispatch is therefore a signed, gated call into the same API/gRPC/MCP fabric that collected the flow, not a free-form command bus. Fig. 2 explains the validation steps agentic actions.

e) *Audit logs*.: Every hop—SOC notify, retrieval, whitelist miss, plan mismatch, human approval, successful apply—emits an audit log back to the SOC/Monitoring/core system (dashed return on the figure). The core archive is the operator-visible history; the ledger holds the immutable receipts. The contract cannot hallucinate and cannot be prompted. This layer guarantees that only a whitelisted, bound, integrity-checked action reaches an agent, not that the LLM was correct.

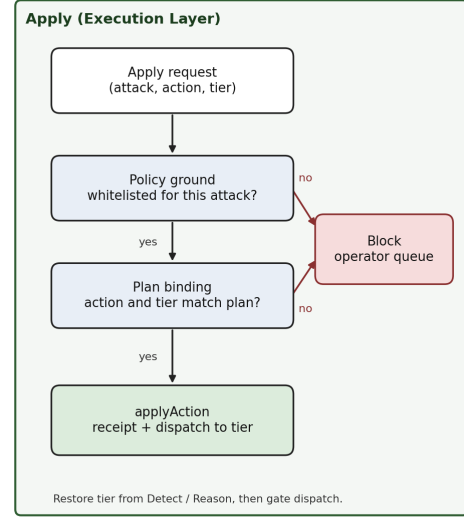


Fig. 2: Apply restores that tier and admits the action only if it is whitelisted and identical to the committed plan.

#### IV. EXPERIMENTAL SETUP AND IMPLEMENTATION

We evaluate our system on CICIDS-2017 [1] using the complete Detect–Reason–Commit–Apply pipeline described in Section III. The evaluation covers three aspects: (i) intrusion-detection performance of the three-party VFL model, (ii) apply-time enforcement against modified mitigation actions, and (iii) blockchain processing overhead. The implementation integrates PyTorch-based VFL and SHAP, a RAG-grounded LLM planner, MySQL/JSON persistence, and the `AgenticTrustRegistry` smart contract into a single executable stack.

##### A. Dataset and Preprocessing

The experiments use a prepared CICIDS-2017 dataset containing 390,398 labeled network-flow records. After preprocessing, 88 numeric traffic features are retained for model development. Non-predictive identifiers such as flow identifiers, source/destination addresses, and timestamps are excluded when present, and only numeric flow attributes are supplied to the learning pipeline.

The original CICIDS attack labels are consolidated into nine classes: BENIGN, BOT, DDOS, DOS, FTTPATATOR, OTHERS, PORTSCAN, SSHPATATOR, and WEBATTACK. Related attack variants are merged into their common parent classes; for example, the DoS variants are grouped as DOS, while Web Attack variants are represented as WEBATTACK. Classes containing fewer than 200 observations are grouped into OTHERS; in the prepared dataset, INFILTRATION (122 flows) and HEARTBLEED (70 flows) form this category.

The resulting data are divided using stratified sampling with random seed 42. We use 64% for training (249,854 flows), 16% for validation (62,464 flows), and 20% for testing (78,080 flows). Stratification preserves the class distribution across all

three splits. Features assigned to each VFL participant are standardized using `StandardScaler` and converted to 32-bit floating-point tensors before training.

### B. VFL Model and Training Configuration

The VFL detector is organized across three enterprise network domains: Access, Perimeter, and Endpoint. Each domain retains its own subset of flow features and processes them locally using an MLP encoder with architecture  $d_p \rightarrow 128 \rightarrow 64$ , where  $d_p$  denotes the number of features available to domain  $p$ . Only the resulting 64-dimensional embeddings are shared with the coordinator; raw feature values remain local. The three embeddings are concatenated into a 192-dimensional representation and passed to a shared classifier with architecture  $192 \rightarrow 128 \rightarrow 9$  to predict one of the nine traffic classes.

The model is trained using weighted cross-entropy loss and the Adam optimizer with a learning rate of  $10^{-3}$  and weight decay of  $10^{-4}$ . Training is limited to 100 epochs with early stopping (patience 20, minimum improvement 0.001), while validation is performed every five epochs. The same data split and training procedure are used for the centralized baseline to ensure a consistent comparison.

### C. Explainability and RAG Configuration

After detection, KernelSHAP is applied to the fused 192-dimensional representation to obtain domain- and feature-level contributions. The predicted class, confidence, and SHAP evidence are then passed to the RAG pipeline. Security documents are split into chunks of 512 with an overlap of 64, embedded using `all-MiniLM-L6-v2`, and indexed in FAISS. The top-5 retrieved passages are reranked using a cross-encoder and provided with the SHAP context to GPT-4o-mini, which generates a structured mitigation plan constrained by the predefined action catalog.

### D. Blockchain and Implementation Stack

The governance component is implemented through `AgenticTrustRegistry.sol` using Solidity 0.8.20. Experiments use a local Hardhat 2.22 Ethereum-compatible network with chain ID 31337. The contract is compiled using `solc` 0.8.24 with optimization enabled for 200 runs, and the backend interacts with the ledger through `Web3.py` and JSON-RPC.

After Reason produces a mitigation plan, the complete artifact remains off-chain in MySQL and JSON storage. The backend canonicalizes the corresponding payload  $P$  and computes

$$c = \text{SHA256}(P).$$

Only the resulting `bytes32` commitment is stored on-chain through `anchor`. Raw network features, SHAP evidence, retrieved policy passages, and human-readable LLM output are therefore not stored on the blockchain. The contract additionally maintains the per-attack action whitelist and the state of successfully applied actions.

TABLE IV: Host and Implementation stack.

| Component    | Configuration                                            |
|--------------|----------------------------------------------------------|
| OS / CPU     | Windows 10 Pro 64-bit; Intel i5-8365U, 4C/8T, 1.60 GHz   |
| RAM / GPU    | 16 GB; Intel UHD 620; no CUDA                            |
| Runtime      | Python 3.11+; Node.js 22                                 |
| Detection    | PyTorch; VFL; KernelSHAP                                 |
| Database     | MySQL 8; JSON artifact storage                           |
| RAG          | LangChain; FAISS; MiniLM; cross-encoder                  |
| LLM          | GPT-4o-mini ( $T \approx 0.3$ )                          |
| Backend / UI | FastAPI / React                                          |
| Contract     | AgenticTrustRegistry, Solidity 0.8.20                    |
| Compiler     | <code>solc</code> 0.8.24; optimizer, 200 runs            |
| Blockchain   | Hardhat 2.22; chain ID 31337                             |
| Client       | <code>Web3.py</code> ; JSON-RPC                          |
| Commitment   | SHA-256 of canonical JSON; <code>bytes32</code> on-chain |

At Apply, `getCommitment` retrieves the stored commitment and the persisted payload is re-hashed before an action is admitted. The execution path then applies the two checks defined in Eq. (1): the action must be permitted by the whitelist for the detected attack class and must belong to the plan committed for the current detection cycle. A successful unit is recorded through `applyAction`.

### E. System Evaluation

The system is evaluated end-to-end using 500 CICIDS-style network flows submitted sequentially through the live backend API. Each flow passes through the Detect–Reason–Commit–Apply pipeline while the trained VFL model, FAISS index, MySQL database, and Hardhat blockchain remain active throughout the run.

To test enforcement integrity, selected actions are modified after Commit but before Apply. The modifications include either actions that are invalid for the detected class or whitelist-valid actions that were not part of the committed plan. Apply then evaluates the whitelist and plan-binding gates to determine whether the modified actions are rejected.

The current executor records execution receipts rather than invoking a live network device. A production deployment can replace this stub with an API, MCP interface, SOAR playbook, or SDN controller without changing the validation gates. The implementation is available at <https://github.com/sharif2008/pragma>.

## V. EXPERIMENTAL RESULTS AND ANALYSIS

We evaluated the proposed system in two settings: detection quality and enforcement performance, including execution behavior and blockchain latency. Detection quality was measured on the held-out CICIDS-2017 test set ( $N=78,080$  flows across nine classes). As shown in Table V, the three-party Vertical Federated Learning (VFL) model achieves an accuracy of 0.9628, macro-recall of 0.9861, and macro-F1 of 0.8633, compared with 0.9614, 0.9829, and 0.8617, respectively, for the centralized MLP trained on all 88 features. The macro-F1 difference is only 0.0016, indicating that feature locality is achieved without materially degrading detection performance.

TABLE V: Detection Performance on CICIDS-2017 (78,080 Samples, 9 Classes)

| Model                               | Acc.    | Mac. Rec. | Mac. F1 |
|-------------------------------------|---------|-----------|---------|
| VFL (3 parties, strict locality)    | 0.9628  | 0.9861    | 0.8633  |
| Standard NN (centralized, 88 feat.) | 0.9614  | 0.9829    | 0.8617  |
| Difference (VFL – NN)               | +0.0014 | +0.0032   | +0.0016 |

The objective of vertical partitioning is therefore privacy-preserving feature locality rather than an accuracy gain over centralized learning.

We ingested 500 network flows through the live API, drawn as a 5% stratified sample per class from a pool of 1,000 records, ensuring proportional representation across all nine attack labels listed in Table VI. Before the first flow was processed, the API was initialized with the trained three-party VFL model, MySQL, FAISS, and a local Hardhat node (chain ID 31337) with a seeded `AgenticTrustRegistry`. Each CICIDS-style feature vector was submitted over HTTP, one flow at a time, emulating how a monitoring component would ingest individual sensor events through `attack_monitor.py`. The experiment therefore evaluates online, row-by-row processing of the deployed stack rather than packet capture from a live network interface.

For each flow, Detect scores the input using the VFL model, applies KernelSHAP, and maps the predicted label to its corresponding policy in Table III. Reason retrieves the relevant RAG context, after which GPT-4o-mini generates a structured mitigation plan containing an average of 3.08 action units per flow. Commit canonicalizes the resulting plan and stores its SHA-256 digest on-chain through `anchor`, creating a tamper-evident commitment to the proposed actions.

For the enforcement experiment, the 500-flow run was executed with `--tamper` enabled. Before Apply, the client modifies either one or two action units per flow, alternating according to row index while leaving the remaining actions unchanged. Each modification represents one of two threat cases derived from Table III: either an action permitted for another attack class but not for the current class, or an action that remains valid under the current class whitelist but was not part of the committed plan. Apply submits the resulting mixed action list, after which the two gates in Eq. (1) are evaluated sequentially: first the class-specific whitelist check and then the plan-binding check. An action unit that passes both gates is recorded through `applyAction`.

Table VI reports the execution outcome for each attack class. Across the 500 flows, the Reason stage generated 1538 action units, of which 750 were deliberately modified. Gate 1 rejected 352 off-whitelist substitutions with `action_not_whitelisted`, while Gate 2 rejected the remaining 398 substitutions with `action_plan_mismatch`. Thus, all 750 modified units were rejected, while the 788 unchanged units successfully received execution receipts. The results demonstrate the complementary roles of the two en-

TABLE VI: Per-class execution outcome on 500 online flows with `--tamper`. Every changed unit is refused (Block.% = 100). WL = whitelist reject; Plan = plan-list reject; Applied = unmodified receipts.

| Attack     | <i>n</i> | Proposed | Changed | WL  | Plan | Applied | Block.% |
|------------|----------|----------|---------|-----|------|---------|---------|
| BENIGN     | 41       | 82       | 60      | 29  | 31   | 22      | 100     |
| BOT        | 37       | 87       | 59      | 27  | 32   | 28      | 100     |
| DDOS       | 35       | 107      | 55      | 26  | 29   | 52      | 100     |
| DOS        | 126      | 357      | 185     | 80  | 105  | 172     | 100     |
| FTPPATATOR | 37       | 188      | 56      | 27  | 29   | 132     | 100     |
| OTHERS     | 60       | 183      | 91      | 47  | 44   | 92      | 100     |
| PORTSCAN   | 31       | 90       | 49      | 23  | 26   | 41      | 100     |
| SSHPATATOR | 30       | 148      | 46      | 20  | 26   | 102     | 100     |
| WEBATTACK  | 103      | 296      | 149     | 73  | 76   | 147     | 100     |
| Total      | 500      | 1538     | 750     | 352 | 398  | 788     | 100     |

TABLE VII: Blockchain time on 500 online flows. Store is Commit (`anchor`); Verify is Apply (`getCommitment`). LLM/RAG and Detect are omitted.

| Stage          | Mean (ms) | $\sigma$ (ms) | Median (ms) | Max (ms) |
|----------------|-----------|---------------|-------------|----------|
| Commit (store) | 45.08     | 31.02         | 40.60       | 653.72   |
| Apply (verify) | 33.83     | 37.70         | 26.58       | 772.56   |
| Store + verify | 78.91     | 48.59         | 69.73       | 812.58   |

forcement gates: Gate 1 prevents actions that violate class-specific policy, whereas Gate 2 detects substitutions that remain policy-valid but differ from the committed mitigation plan.

In the testbed, the final executor is implemented as a stub that records only the execution receipt. In a production SOC environment, this component can be replaced by an operational interface such as an API call, MCP tool, SOAR playbook, or SDN enforcement hook, while the whitelist and plan-binding gates remain unchanged.

Table VII summarizes the blockchain processing overhead across the 500 online flows, excluding Detect, RAG retrieval, and LLM reasoning. Commit stores the SHA-256 plan commitment using `anchor`, requiring an average of 45.08 ms per flow, while Apply retrieves and verifies the stored commitment using `getCommitment` in 33.83 ms. Together, these operations introduce an average blockchain overhead of 78.91 ms per flow. The higher Commit latency is consistent with the additional cost of an on-chain write compared with commitment retrieval and verification. Overall, the results show that the proposed system adds modest blockchain overhead while providing tamper-evident plan integrity, auditability, and controlled enforcement of agent-generated actions.

## VI. DISCUSSION

Our proposed system processes each network flow through four stages: Detect, Reason, Commit, and Apply. Detect uses vertical federated learning, allowing each party to retain its local feature columns without sharing raw flow data. Reason grounds mitigation planning in RAG-retrieved guidance from public sources such as NIST, CIS, and MITRE ATT&CK. Commit anchors a SHA-256 digest of the generated action plan, while Apply enforces two checks before execution: each

action must be permitted by the class-specific whitelist and must match the committed plan. If an action is modified before Apply, either to one that is invalid for the predicted class or to a valid but previously uncommitted action, that unit is rejected, while unchanged valid units may still proceed.

The current testbed has several limitations. All VFL parties are implemented as column partitions on a single machine, so WAN latency, communication failures, and network partitions are not evaluated. The blockchain component runs on a local Hardhat node rather than a production ledger, and the Apply stage records execution receipts without invoking a live firewall, SOAR platform, or SDN controller. Nevertheless, the testbed preserves the deployment structure required for a distributed implementation: locally retained features, policy-grounded mitigation planning, and blockchain-backed validation before execution. A production deployment could therefore retain the Detect and Reason components, replace the local ledger with a production blockchain, and connect the Apply stage to a real enforcement interface after both validation gates succeed.

## VII. CONCLUSION

This paper presented a privacy-preserving and verifiable intrusion-response architecture that connects collaborative detection with controlled mitigation execution. The proposed Detect–Reason–Commit–Apply workflow enables multiple network domains to participate in threat detection while retaining local data ownership and introduces deterministic safeguards before generated mitigation actions are allowed to proceed.

The evaluation shows that the proposed VFL detector achieves performance comparable to centralized learning while preserving feature locality. More importantly, the enforcement mechanism successfully distinguishes valid committed actions from unauthorized or modified actions, demonstrating that LLM-generated mitigation plans can be subjected to explicit policy and integrity checks before execution. These results support the feasibility of combining federated detection, grounded reasoning, and blockchain-backed enforcement to improve accountability in agentic network security.

Overall, the proposed system provides a foundation for trustworthy autonomous intrusion response in multi-domain environments, where privacy, explainability, and controlled execution must be maintained alongside detection accuracy. Future work will evaluate the system in distributed multi-site deployments, measure communication and blockchain overhead, and replace the local Hardhat node with a production-grade permissioned ledger. It will also assess VFL robustness against adversarial feature manipulation, evasion attacks, and poisoned inputs.

## REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018.
- [2] National Institute of Standards and Technology, "Security and privacy controls for information systems and organizations," NIST SP 800-53 Rev. 5, 2020.
- [3] National Institute of Standards and Technology, "Zero trust architecture," NIST SP 800-207, 2020.
- [4] Y. Liu et al., "Vertical federated learning: Concepts, advances, and challenges," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3615–3634, Jul. 2024.
- [5] F. Folino et al., "A scalable vertical federated learning framework for analytics in the cybersecurity domain," in *Proc. 32nd Euromicro Int. Conf. PDP*, Dublin, Ireland, 2024, pp. 245–252.
- [6] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023.
- [7] T. Li and Q. Zhu, "Agentic AI for cyber resilience: A new security paradigm and its system-theoretic foundations," arXiv:2512.22883, 2025.
- [8] S. Shajarian, K. Marsh, J. Benson, S. Khorsandroo, and M. Abdelsalam, "ReGAIN: Retrieval-grounded AI framework for network traffic analysis," in *Proc. IEEE ICNC*, 2026.
- [9] T. Fu, J. Hu, G. Min, S. A. Khawaja, K. Singh, and K. Dev, "Federated retrieval-augmented generation-based LLM for enhanced cyber threat detection in the Internet-of-Energy," *IEEE Network*, 2026.
- [10] MITRE Corporation, "MITRE ATT&CK: A knowledge base of adversary tactics and techniques," 2024.
- [11] M. M. Karim, D. H. Van, S. Khan, Q. Qu, and Y. Kholodov, "AI agents meet blockchain: A survey on secure and scalable collaboration for multi-agents," *Future Internet*, vol. 17, no. 57, Feb. 2025.
- [12] B. Chen, G. Li, X. Lin, Z. Wang, and J. Li, "BlockAgents: Towards Byzantine-robust LLM-based multi-agent coordination via blockchain," in *Proc. ACM Turing Award Celebration Conf.*, Changsha, China, 2024, pp. 187–192.
- [13] A. Jin, Y. Ye, B. Lee, and Y. Qiao, "DeCoAgent: Large language model empowered decentralized autonomous collaboration agents based on smart contracts," *IEEE Access*, vol. 12, pp. 155234–155245, 2024.
- [14] S. Jan, H. A. Razzaqi, A. Akarma, and M. R. Belgaum, "A blockchain-monitored agentic AI architecture for trusted perception–reasoning–action pipelines," in *Proc. IEEE ICCA*, Bahrain, 2025.
- [15] M. Alazab et al., "Federated learning for cybersecurity: Concepts, challenges, and future directions," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3501–3509, May 2022.
- [16] J. Talpini et al., "SHIELDED: A network-aware approach for secure and trustworthy federated learning," *IEEE Network*, 2025.
- [17] R. Amin et al., "An efficient federated learning based defense mechanism for software defined network cyber threats through machine learning models," *Scientific Reports*, vol. 15, p. 41390, 2025.
- [18] L. Yu, M. Han, Y. Li, C. Lin, Y. Zhang, M. Zhang, Y. Liu, H. Weng, Y. Jeon, K.-H. Chow, and S. Patterson, "A survey of privacy threats and defense in vertical federated learning: From model life cycle perspective," arXiv:2402.03688, Feb. 2024.
- [19] M. Ye, W. Shen, B. Du, E. Snezhko, V. Kovalev, and P. C. Yuen, "Vertical federated learning for effectiveness, security, applicability: A survey," arXiv:2405.17495, Jun. 2024.
- [20] N. Kshetri, "Transforming cybersecurity with agentic AI to combat emerging cyber threats," *Telecommunications Policy*, vol. 49, p. 102976, 2025.
- [21] M. A. Alsawaiet, "ZeroDay-LLM: A large language model framework for zero-day threat detection in cybersecurity," *Information*, vol. 16, no. 11, art. 939, 2025.
- [22] S. J. Lazer, K. Aryal, M. Gupta, and E. Bertino, "A survey of agentic AI and cybersecurity: Challenges, opportunities and use-case prototypes," arXiv:2601.05293, Jan. 2026.
- [23] A. Chhabra, S. Datta, S. K. Nahin, and P. Mohapatra, "Agentic AI security: Threats, defenses, evaluation, and open challenges," *IEEE Access*, vol. 14, pp. 49455–49482, 2026.
- [24] N. R. Pendli, "Securing agentic AI systems through blockchain: A comprehensive review of trust, autonomy, and decentralized frameworks," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 9s, 2025.
- [25] S. Alqithami, "Autonomous agents on blockchains: Standards, execution models, and trust boundaries," arXiv:2601.04583, 2026.
- [26] B. Chen, G. Li, J. Wu, J. Li, M. Chen, and J. Wang, "AgentChain: Blockchain-empowered multi-agent coordination for trustworthy LLM question-answering systems," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 4, pp. 8501–8518, 2026.
- [27] T. A. Syed, M. R. Belgaum, S. Jan, A. A. Khan, and S. S. Alqahtani, "Agentic AI for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation," arXiv:2512.23480, 2025.

- [28] G. Wickremasinghe, F. R. Sandjaja, K. Rafferty, and V. Sharma, "State-verification in asynchronous blockchain-driven agentic AI systems," in *Proc. 40th Int. Conf. Information Networking (ICOIN)*, IEEE, 2026, pp. 139–144.
- [29] M. Rahmati Tabrizi, A. Karami, and A. Aleva, "BEAF: Blockchain-enabled accountability framework for high-assurance clinical AI," 2025.
- [30] Y. Gao et al., "Skillsets on the chain: A blockchain-based trustworthy agentic AI networking framework," 2025.
- [31] M. Fan et al., "SecureVFL: Privacy-preserving multi-party vertical federated learning based on blockchain and RSS," *Digital Communications and Networks*, vol. 11, no. 5, pp. 837–849, 2025.
- [32] A. Singh, V. Pareek, and A. Sharma, "Blockchain-enabled zero trust framework for securing FinTech ecosystems against insider threats and cyber attacks," 2025.
- [33] P. Zhang, S. Ding, and Q. Zhao, "Exploiting blockchain to make AI trustworthy: A software development lifecycle view," *ACM Computing Surveys*, vol. 56, 2024.
- [34] M. Xu, "The agent economy: A blockchain-based foundation for autonomous AI agents," arXiv:2602.14219, 2026.
- [35] D. Ressi, R. Romanello, C. Piazza, and S. Rossi, "AI-enhanced blockchain technology: A review of advancements and opportunities," *Journal of Network and Computer Applications*, vol. 225, Art. 103858, 2024.
- [36] B. Nguyen Thanh, H. X. Son, and D. T. H. Vo, "Blockchain: The economic and financial institution for autonomous AI?" *Journal of Risk and Financial Management*, vol. 17, no. 54, 2024.
- [37] Y. Wu, N. Xing, G. Chen, T. T. A. Dinh, Z. Luo, B. C. Ooi, X. Xiao, and M. Zhang, "Falcon: A privacy-preserving and interpretable vertical federated learning system," *Proc. VLDB Endowment*, vol. 16, no. 10, pp. 2471–2484, 2023.
- [38] Center for Internet Security, "CIS Controls Implementation Guide," Version 8.1.2, 2025.